ROVER PROJECT · YDLIDAR X2 · OBSTACLE STOP

lidar_stop: how the code works

A walk-through of `lidar_stop.cpp`, from a spinning laser to the word **STOP**. Written with the Feynman technique: simple words first, then the code.

IN ONE SENTENCE
What the part does, in words a 12-year-old follows.

ANALOGY
The same idea from everyday life, so it sticks.

GAP / CAREFUL
Where it breaks, or what people get wrong.

CHECK YOURSELF
Questions to answer out loud. If you can't, reread that part.

1. The whole program in one page
2. How the X2 measures distance
3. What happens when you run it
4. Config: the settings panel
5. ScanSource: real LiDAR or simulator
6. analyze(): the heart
7. StopLatch: stop fast, resume carefully
8. drawRadar(): seeing what it thinks
9. main(): gluing it together
10. One scan, start to finish
11. Setup, calibration, troubleshooting
12. Known limits
13. Explain it back (self-test)
14. Glossary

1. The whole program in one page

IN ONE SENTENCE

About 7 times a second, the program gets a full 360° ring of distance readings from the LiDAR, keeps only the trustworthy points, checks whether any of them are in the strip of floor the rover is about to drive over, and says STOP if something is closer than 30 cm.

ANALOGY

Walking through a dark room with your arms stretched out in front, as wide as your body. You don't care about the wall two metres to your left. You only care whether something is about to touch your outstretched hands. The moment something does, you stop. You only walk on once you're sure it has moved away, not the instant you stop feeling it.

#	Part (in the code)	Its one job	Lines
1	`struct Config`	Holds every number: LiDAR settings, rover size, stop distances	47–75
2	`class ScanSource` + `X2Source` + `SimSource`	Delivers one 360° scan at a time, from the real X2 or from a fake room	85–223
3	`dropLonelyPoints()` + `analyze()`	Turns raw readings into "obstacle ahead at X m, free space left/right"	242–302
4	`class StopLatch`	Decides STOP or CLEAR without flickering	309–327
5	`drawRadar()`	Draws a top-down radar picture so you can see what it thinks	332–366
—	`main()`	Reads your options, picks real or simulated LiDAR, runs the loop	371–429

One scan flows left to right, ~7 times a second

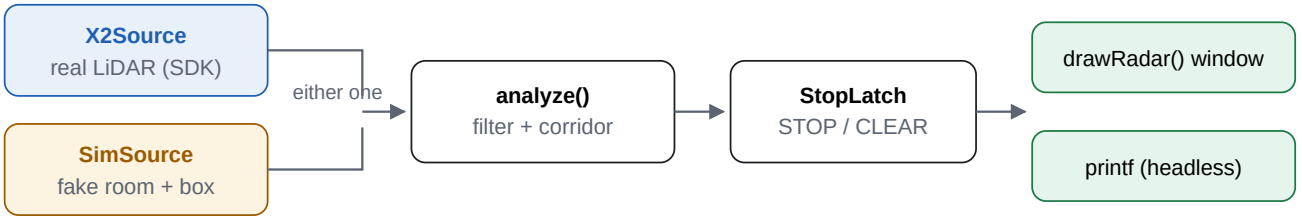


Figure 1. The program's parts. The rest of the code doesn't care whether scans come from the real X2 or the simulator.

2. How the X2 measures distance

IN ONE SENTENCE

The X2 shines a laser dot, watches where the dot lands on a tiny camera sensor a few centimetres away, and works out the distance from that position. It spins so it does this all the way around.

ANALOGY

Hold your thumb up and look at it with one eye, then the other: it jumps sideways. A near thumb jumps a lot, a far one barely moves. The X2 does the same with a laser dot and a camera placed side by side: how far the dot "jumps" on the sensor tells it the distance. This is called **triangulation**.

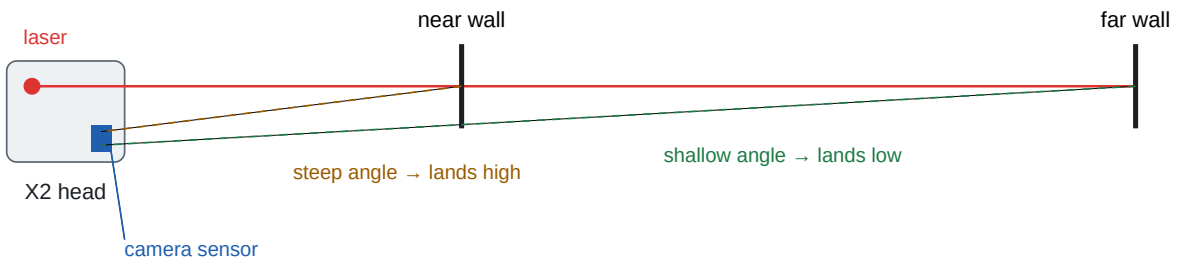


Figure 2. Triangulation. The reflected dot lands at a different spot on the sensor depending on distance.

Numbers worth knowing (from the YDLidar SDK's device table and the scan data):

Property	X2 value	What it means for us
Range	0.10–8.0 m	Anything under 10 cm is unreliable, so we throw it away
Spin rate	~7 Hz (4–8 Hz)	A fresh 360° picture every ~143 ms. Set by motor voltage, not software
Samples	3,000 per second	≈ 430 points per rotation
Angular gap	360° ÷ 430 ≈ 0.84°	Neighbouring points are 1.5 cm apart at 1 m, 4.4 cm apart at 3 m
Baud rate	115200	Must match, or you get garbage or nothing
Communication	One-way ("single channel")	The X2 only talks and ignores commands, so you can't set its speed from code
Scan shape	One flat plane	It only sees things at its own height

CAREFUL

Triangulation gets less precise with distance. The dot's position changes less and less as things get farther away, like the thumb that barely jumps. That's fine for us: we care most about the first metre.

CHECK YOURSELF

Why can't the code set the X2's spin speed? How far apart are two neighbouring points on a wall 2 m away? (Hint: 2 m × 0.84° in radians.)

3. What happens when you run it

> **IN ONE SENTENCE**
> It reads your options, starts the LiDAR (or the simulator), then loops: get a scan, analyse it, decide STOP or CLEAR, show it, until you press `q` or Ctrl+C.

```
./lidar_stop --port /dev/ttyUSB0 --offset -90
```

#	What happens	Where
1	Reads options: port, forward offset −90°	`main()`, the `for` loop over `argv`
2	Makes `printf` print each line straight away	`setvbuf(stdout, …, _IOLBF, 0)`
3	Installs the SDK's Ctrl+C handler, so Ctrl+C stops the loop cleanly instead of killing the program with the motor running	`os_init()`
4	Creates an `X2Source` (or a `SimSource` with `--sim`)	`std::make_unique<…>`
5	Configures the X2 (baud, one-way, sample rate…), connects and starts the motor	`X2Source::start()`
6	**Loop**: wait for the next full rotation	`src->next(scan, hz)`
7	Filter points, find the nearest obstacle ahead and the side clearances	`analyze()`
8	STOP or CLEAR, with hysteresis	`latch.update()`
9	Draws the radar window (or prints a line in `--headless`)	`drawRadar()` / `printf`
10	On quit: stop the motor and disconnect	`src->stop()`

4. Config: the settings panel

> **IN ONE SENTENCE**
> One box holding every number, in three groups: things fixed by the X2, things you measure on your rover, and things that decide behaviour.

Setting	Default	What it means	Change it when…
Fixed by the X2: don't touch			
`baud`, `sampleRateK`, `singleChannel`	115200, 3, true	The X2's communication settings, from the SDK's device table.	Never, unless you change LiDAR model.
`minRangeM`, `maxRangeM`	0.10, 8.0	Readings outside this range are thrown away.	Never.
Measure on your rover			

`forwardOffsetDeg`, `mirror`	0, false	Rotates (and optionally flips) angles so 0° is your rover's front and left is positive.	Always, via `--calib` (section 11).
`lidarToFrontM`, `lidarToRearM`	0.08, 0.15	From the LiDAR's centre to the front bumper / rear edge.	Measure with a ruler.
`halfWidthM`	0.10	Half the rover's width.	Measure.
`marginM`	0.05	Extra safety width on each side of the corridor.	It clips things at the sides: raise it. It stops for things beside the path: lower it.
Behaviour			
`stopDistM`	0.30	STOP when an obstacle is closer than this, measured from the **front bumper**.	The rover is faster (raise) or slower (lower).
`resumeDistM`	0.40	Only CLEAR once it's farther than this.	Keep it about 10 cm above `stopDistM`.
`neighbourTolM`, `neighbourMaxDeg`	0.05, 3	Lonely-point filter: a point is kept only if a neighbour lies within 5 cm and 3°.	Real objects get dropped: loosen. Noise gets through: tighten.
`kthNearest`	2	Uses the 2nd-closest point in the corridor, not the closest.	Rarely.
`scansToStop`, `scansToResume`	1, 5	Scans in a row needed to stop / to resume.	Too many false stops: `scansToStop = 2`.
`sideMinDeg`, `sideMaxDeg`	30, 100	The angle sectors used to measure free space on each side.	For the future bypass feature.

---

5. ScanSource: real LiDAR or simulator

> **IN ONE SENTENCE**
>
> `ScanSource` is a promise: "I can `start()`, give you the `next()` scan, and `stop()`". Two classes keep that promise in different ways: one talks to the real X2, the other invents a fake room.

> **ANALOGY**
>
> A wall socket. Your laptop charger doesn't care whether the electricity comes from a power station or a generator, as long as the plug fits. `ScanSource` is the socket shape. `X2Source` and `SimSource` are two power sources that fit it. `analyze()` is the laptop: it just uses what comes out.

```
class ScanSource {                    // the "socket shape" (an interface)
    virtual bool start() = 0;          // "= 0" means: every child MUST write this
    virtual bool next(vector<pair<float,float>>& out, float& scanHz) = 0;
    virtual void stop() {}             // optional: does nothing unless overridden
};
std::unique_ptr<ScanSource> src = sim ? make_unique<SimSource>() : make_unique<X2Source>(cfg);
```

This C++ feature is called **polymorphism** (`virtual` functions). `unique_ptr` is a smart pointer: it deletes the object automatically when `main()` ends, so you never write `delete`. Each scan is a list of `(angle in radians, distance in metres)` pairs.

`X2Source::start()`: setting up the real X2

It calls `laser_.setlidaropt(Option, &value, size)` once per setting, then `initialize()` (open the port, check the device) and `turnOn()` (start the motor and the SDK's background reading thread).

SDK option	Value	Why
`LidarPropSerialBaudrate`	115200	X2's speed
`LidarPropLidarType`	`TYPE_TRIANGLE`	The X2 measures by triangulation (section 2)
`LidarPropSampleRate`	3	3,000 samples per second
`LidarPropSingleChannel`	true	One-way communication; the SDK won't wait for replies that never come
`LidarPropInverted`	**true**	Angles counter-clockwise, left positive: the maths and ROS convention. The SDK's default is clockwise.
`LidarPropMinAngle` / `MaxAngle`	−180 / 180	Keep the full circle
`LidarPropMinRange` / `MaxRange`	0.10 / 8.0	X2 spec
`LidarPropAutoReconnect`	true	If the cable jiggles, the SDK reconnects by itself
`LidarPropScanFrequency`	7	Only a hint. The X2's real speed comes from motor voltage.

`next()` calls `laser_.doProcessSimple(scan)`, which waits until one full rotation has arrived, then copies each point's angle and range into the output list. `stop()` calls `turnOff()` (motor off) and `disconnecting()` (close the port).

`SimSource`: a fake room for testing at your desk

It pretends to be an X2 so you can test all the logic without hardware. Every 140 ms (about 7 Hz) it produces 430 readings of a 3 m × 2.4 m room with a 20 cm box that drives towards the rover, sits in front of it, then slides away to the left, repeating every 14 s. To make it realistic, it adds:

- **Noise:** about ±8 mm random error on every reading
- **Dropouts:** 3% of readings come back as 0 m, meaning "no return"
- **Dust:** 0.4% of readings are fake points 12–22 cm away

For each simulated laser ray, it computes where the ray hits the walls (`wall()`) and the box (`rayBox()`, using the "slab method"), and keeps whichever is nearer, just as a real laser stops at the first thing it hits.

> **CHECK YOURSELF**
> Why is `LidarPropInverted` set to true? What would break if `SingleChannel` were false on an X2? Why is a simulator worth having if you already own the LiDAR?

6. analyze(): the heart

> **IN ONE SENTENCE**
> Takes one raw scan of about 430 (angle, distance) pairs and returns: the distance from the front bumper to the nearest real obstacle in the rover's path, the free space on the left and right, and the points to draw.

Step 2: drop invalid readings

```
if (range >= 0.10 && range <= 8.0) keep it;     // 0 m = "no return", <10 cm = too close to trust
```

Step 2b: `dropLonelyPoints()`, "real objects are solid"

> **IN ONE SENTENCE**
> Keeps a point only if at least one of its nearby points, going round the circle, is at almost the same distance.

> **ANALOGY**
> One person in a group says they saw a ghost; nobody else did. You don't believe it. If three people standing next to each other all saw the same thing, you do. A real box reflects the laser at several neighbouring angles, all at about the same distance. A speck of dust gives one point, alone.



Figure 3. The lonely-point filter. Groups survive, even pairs like a chair leg; single stray points don't.

```
sort points by angle;
for each point i:
    for d in {-2, -1, +1, +2}:                          // look 2 neighbours each side
        j = (i + d + n) % n;                            // "% n" wraps around: after +180° comes -180°
        if (angle gap ≤ 3°  &&  |range_j - range_i| < 5 cm) → supported, keep i;
```

Why look 2 neighbours each side, not 1? Because of dropouts. If the reading right next to a real point came back as 0 m (already thrown away), the next one along still counts. **Why the 3° limit?** After dropouts are removed, "neighbour in the list" could be far away in angle. The limit makes sure neighbours are really side by side.

A real bug this fixed: in the first version, three dust points in one scan (at −20°, +30° and +41°) caused a false STOP with the box 1.4 m away. None of them had a neighbour, so this filter removes all three.

Step 3: rotate so 0° means "rover forward"

```
deg = rad2deg(angle) + forwardOffsetDeg;     // e.g. -60° + 90° = 30°
if (mirror) deg = -deg;                       // swap left and right
while (deg > 180) deg -= 360;                 // keep it in (-180, 180]
while (deg <= -180) deg += 360;               // e.g. 200° → -160°
```

ANALOGY

A compass that's been glued onto the rover crooked. It still works; you just add a fixed correction to every reading. Calibration (section 11) finds that correction once. The wrap-around is like a clock: 13 o'clock is 1 o'clock.

Step 4: polar to Cartesian (angle and distance to x, y)

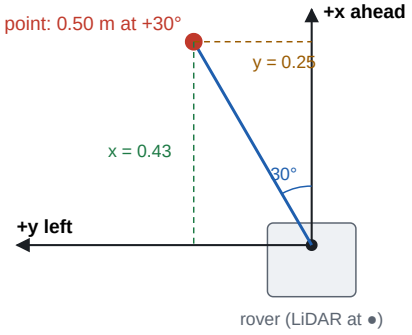


Figure 4. The rover's frame. x points ahead, y points left, and angles are counter-clockwise from ahead.

```
x = range × cos(angle)       // how far AHEAD
y = range × sin(angle)       // how far to the LEFT (negative = right)
```

ANALOGY

"The shop is 500 m away, 30° to the left" (a bearing and a distance) versus "go 433 m forward, then 250 m left" (two straight-line distances). Same place, two ways of saying it. Boxes and corridors are much easier to check in the second form.

Worked examples:

Reading	x = r·cos	y = r·sin	Result
0.50 m at +30°	0.433	+0.250	Left of the corridor (|y| > 0.15), so it isn't in the path. It's in the left sector (30–100°), so left clearance = 0.50 m.
0.35 m at +10°	0.345	+0.061	In the corridor. Ahead = 0.345 − 0.08 = **0.265 m**, under 0.30, so **STOP**.
0.12 m at +150°	−0.104	+0.060	Inside the rover's own outline, so it's the chassis and is ignored (step 5).

Step 5: ignore the rover itself

```
onRover = x > -lidarToRearM && x < lidarToFrontM && |y| < halfWidthM;
```

The laser also hits your own wires, standoffs and chassis. Anything inside the rover's own rectangle is thrown away, like ignoring your own nose in your field of view.

Step 6: the corridor, a rectangle and not a cone

```
inCorridor = x > lidarToFrontM  &&  |y| < halfWidthM + marginM;      // ahead of the bumper,
within 15 cm of centre
ahead      = x - lidarToFrontM;                                     // measured from the BUMPER,
not the LiDAR
```

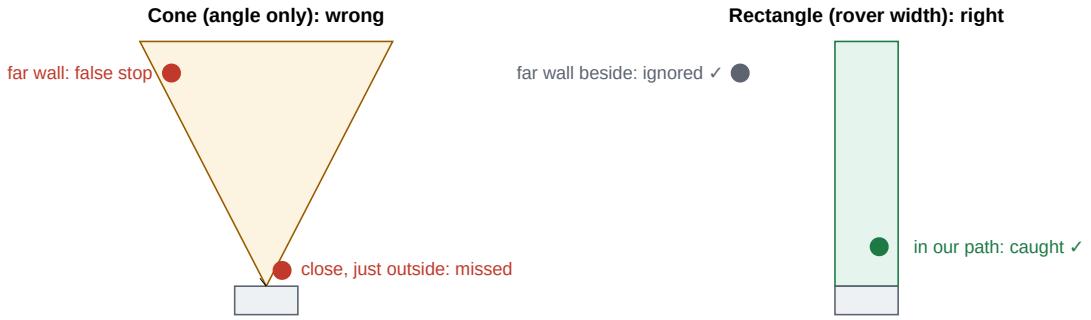


Figure 5. A cone is too wide far away and too narrow close up. A rectangle matches the path the rover's body will actually sweep.

Side sectors: free space left and right

```
if ( 30° ≤ deg ≤ 100°) leftClearM  = min(leftClearM,  range);
if (-100° ≤ deg ≤ -30°) rightClearM = min(rightClearM, range);
```

Not used for stopping. It's the input the bypass feature will need: "which side has more room to drive around the obstacle?"

Step 7: the 2nd-nearest point, not the nearest

```
std::nth_element(ahead.begin(), ahead.begin() + 1, ahead.end());   // partial sort: find the
2nd smallest fast
aheadM = ahead[1];
```

Worked example: the corridor distances are {0.41, 0.40, 0.43, 0.12}, where 0.12 is a stray point that somehow survived the filter. The nearest would say 0.12, a false STOP. The 2nd-nearest says **0.40**, so no STOP. One stray point alone can never stop the rover, but two points (a chair leg) still can. `nth_element` is faster than a full sort: it only guarantees the item at that position is correct, like picking the 2nd-tallest student without lining up the whole class.

CHECK YOURSELF

Convert 0.60 m at −45° to (x, y). Is it in the corridor? (x = 0.424, y = −0.424: no, it's to the right.) Why is `ahead` measured from the bumper and not the LiDAR?

7. StopLatch: stop fast, resume carefully

IN ONE SENTENCE

Remembers whether we're stopped, switches to STOP the moment something is under 30 cm, and only switches back after 5 scans in a row (about 0.7 s) with nothing under 40 cm.

ANALOGY: HYSTERESIS

A room thermostat that turns the heater ON at 18 °C and OFF at 21 °C. If on and off were both at 20 °C, it would click on and off every few seconds as the temperature wobbled around 20. The gap between the two numbers is called **hysteresis**. Here the gap is 30 cm (stop) and 40 cm (resume).

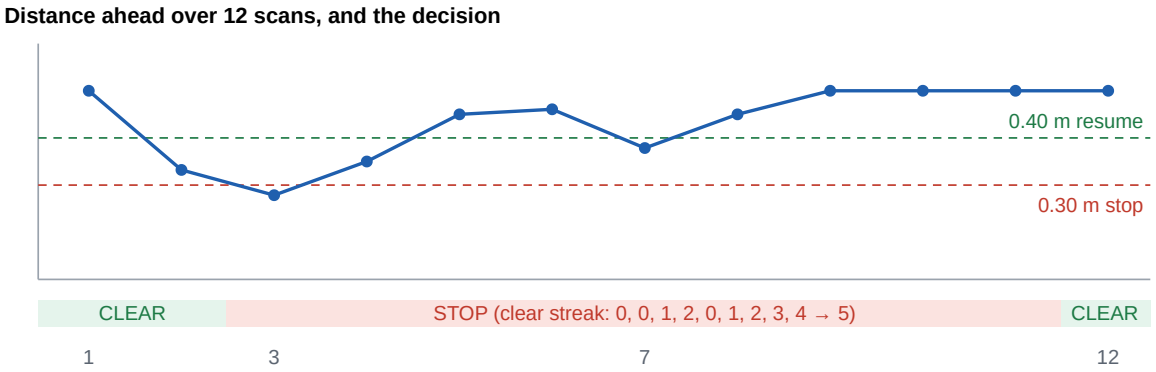


Figure 6. The distance dips under 0.30 at scan 3, so STOP straight away. At scan 4 (0.35) it stays stopped, because it's between the lines. Scan 7 (0.38) resets the count. CLEAR only returns after 5 scans in a row above 0.40.

```
if (!blocked_) {                                          // currently CLEAR
    stopStreak_ = aheadM < 0.30 ? stopStreak_ + 1 : 0;
    if (stopStreak_ >= 1) blocked_ = true;               // scansToStop = 1: react on the first
scan
} else {                                                   // currently STOP
    clearStreak_ = aheadM > 0.40 ? clearStreak_ + 1 : 0;  // any bad scan resets the count
    if (clearStreak_ >= 5) blocked_ = false;
}
```

Why lopsided (1 scan to stop, 5 to resume)? Because the two mistakes don't cost the same. A false stop costs about 0.7 s. A missed stop costs a collision. So the rover believes bad news straight away and good news only after checking.

How fast does it react? A new scan arrives every ~143 ms. In the worst case, an obstacle appears just after a scan, so the rover drives up to ~143 ms more before it knows. At 0.3 m/s that's about 4 cm, well inside the 30 cm stop distance. (Braking distance and the ESP32 link add more; measure the total.)

CHECK YOURSELF

The rover is stopped. The next scans read 0.45, 0.46, 0.39, 0.45, 0.47. Is it CLEAR yet? (No. 0.39 reset the streak, so only 2 clear scans have been counted since.) Why not also use 5 scans to stop?

8. drawRadar(): seeing what it thinks

> **IN ONE SENTENCE**
>
> Draws a top-down map with the rover in the middle and forward pointing up, showing every point, the corridor, the stop line and the decision.

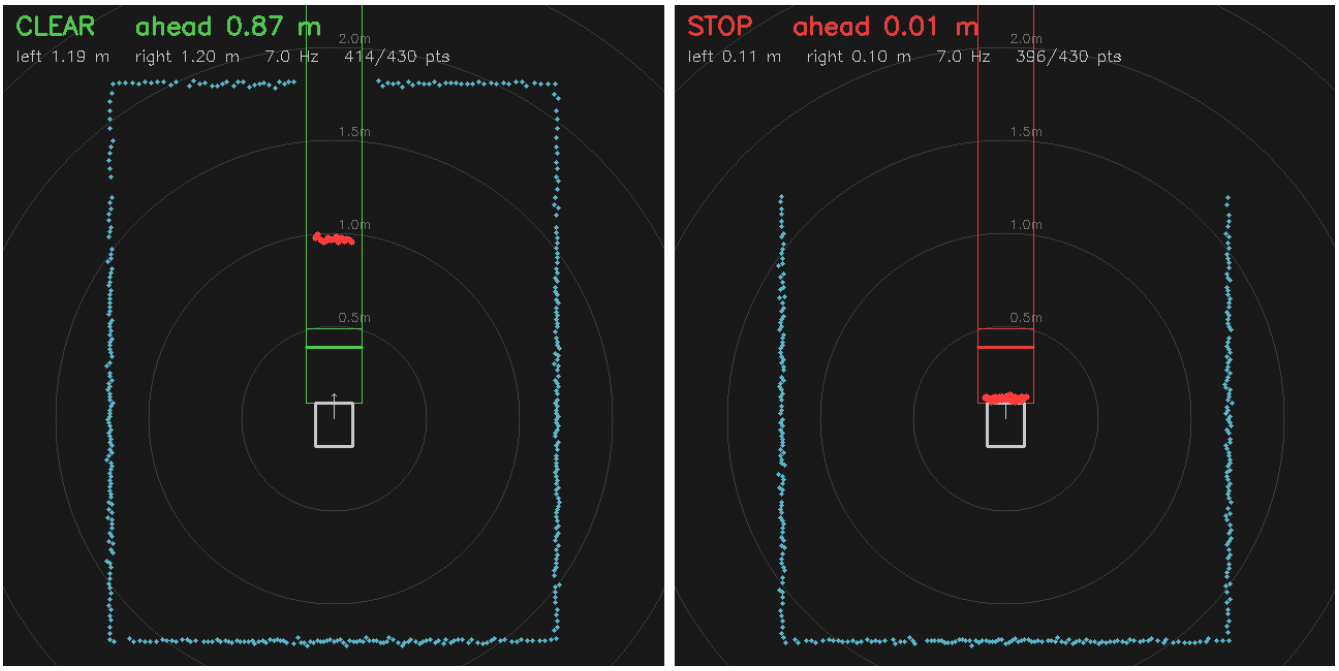


Figure 7. Simulator run. Left: box at 0.87 m, CLEAR. Right: box right at the bumper, STOP. The gap in the far wall on the left is the box's "shadow": a laser can't see through things.

You see	Meaning
Grey rings	Every 0.5 m from the LiDAR
Grey rectangle + arrow	Your rover and its forward direction
Tall box (green or red)	The corridor. It turns red on STOP
Thick line / thin line	Stop line (30 cm) / resume line (40 cm) from the bumper
Blue-yellow dots	Valid points (walls, furniture)
Red dots	Points inside the corridor: these are what trigger stops
Top text	Decision, distance ahead, left/right clearance, spin rate, and points kept out of points received

`toPx(x, y)` converts metres to pixels (1 m = 180 px). x goes *up* the screen, so it subtracts from the pixel row; y goes *left*, so it subtracts from the pixel column. That's why forward is up and left is left, like looking down at the rover from the ceiling.

9. main(): gluing it together

Option	Effect
`--port /dev/ttyUSB0`	Which serial port the X2 is on
`--offset -90`	Forward correction in degrees (from calibration)
`--mirror`	Swap left and right
`--sim`	Use the fake room instead of the LiDAR
`--headless`	No window. Prints a line every 7 scans and on every change
`--calib`	Prints the nearest point's angle, for finding "forward"

- **`os_init()` / `os_isOk()`:** the SDK catches Ctrl+C and makes `os_isOk()` return false, so the loop ends normally and `src->stop()` switches the motor off. Killing a program mid-scan can leave the X2 spinning.
- **If `next()` fails**, it prints "No scan (check cable, port, 5 V power)" and tries again. With auto-reconnect on, a loose cable often recovers by itself.
- **It doesn't talk to the ESP32 yet.** It only prints and draws STOP/CLEAR. The next step merges this with the camera program so one "brain" owns the serial port.

10. One scan, start to finish

#	Happens	Data after this step
1	X2 finishes one rotation, SDK hands it over	~430 (angle, range) pairs
2	Drop 0 m, <0.10 m, >8 m	~415 pairs
2b	Drop lonely points	~410 pairs; dust gone
3–4	Rotate by offset, convert to (x, y)	~410 points in the rover's frame
5	Drop hits on the rover itself	~405 points
6	Corridor test; side sectors	e.g. 12 corridor points; left 1.19 m, right 1.20 m
7	2nd-nearest corridor point	one number, e.g. **0.28 m**
8	StopLatch	**STOP**
9	Radar window / printed line	what you see

11. Setup, calibration, troubleshooting

Calibrate "forward" (2 minutes, do this first)

1. `./lidar_stop --port /dev/ttyUSB0 --calib`
2. Hold your hand about 20 cm in **front** of the rover. It prints e.g. `nearest point: 0.21 m at +90 deg`.

3. Run with the opposite angle: `--offset -90`. Your hand should now appear straight **up** in the radar.
4. Move your hand to the rover's **left**. If it shows on the right of the window, add `--mirror`.
5. Put the final value into `forwardOffsetDeg` in `Config`, then measure the rover (section 4).

Symptom → cause → fix

Symptom	Likely cause	Fix
No `/dev/ttyUSB0`	Cable, or the adapter isn't detected	Replug; check `dmesg | tail` for a CP210x / CH340 line
"Permission denied" on the port	Not in the `dialout` group	`sudo usermod -aG dialout $USER`, then log out and back in
Motor doesn't spin	Not enough 5 V power	Another USB port, a powered hub, or the adapter's extra power port
Spins but "No scan"	Wrong baud or single-channel setting	Must be 115200 and single-channel = true
"Could not find ydlidar_sdk"	SDK not installed	Build the SDK, then `sudo make install`
Stops for things beside the rover	Wrong `--offset`, or corridor too wide	Redo calibration; check `halfWidthM` and `marginM`
Everything close shows as an obstacle	Hits on the rover's own parts	Measure `lidarToFrontM`, `lidarToRearM` and `halfWidthM` properly
Random short stops	Noise that looks like a thin object	`scansToStop = 2`
Motor keeps spinning after exit	Program was killed, not quit	Quit with `q` or Ctrl+C

---

12. Known limits

Feynman's third step is finding the gaps. Say these before someone asks.

- **One flat plane.** The X2 sees only at its own height. A shoe, a cable or a low step below it is invisible. That's exactly why the camera matters.
- **Glass and mirrors** let the laser through or bounce it away, so they can be missed.
- **Very dark or very shiny surfaces** return few points.
- **Thin objects** return only 1–2 points. Two survive the filter; one doesn't.
- **Two neighbouring noise points** at the same distance look like a chair leg and cause a brief false stop. This is a deliberate trade-off (section 7).
- **Reaction time:** up to one scan (~143 ms) plus processing and the ESP32 link. Measure it end-to-end before trusting a stop distance.
- **Moving obstacles:** it only stops and waits. Going around is the next feature (that's what `leftClearM` / `rightClearM` are for).

13. Explain it back (self-test)

1. How does a triangulation LiDAR measure distance?
2. Why is a scan "a list of (angle, distance)", and why convert it to (x, y)?
3. What does the lonely-point filter do, and why does it look 2 neighbours each side?
4. Why a rectangle corridor instead of an angle cone?
5. Why use the 2nd-nearest point in the corridor instead of the nearest?
6. What is hysteresis, and why do we stop after 1 scan but resume after 5?
7. Why is the obstacle distance measured from the bumper, not the LiDAR?
8. What are `ScanSource`, `virtual` and `unique_ptr` doing here?
9. Name two things the X2 can't see, and what covers for it.

Answers

1. It fires a laser dot and watches where the reflection lands on a sensor next to it. The landing position depends on distance, just as a near thumb "jumps" more between your eyes. It spins to do this all the way round.
2. The LiDAR naturally reports "in this direction, this far". Rover questions like "is it within 15 cm of our centre line?" are easy in x, y and awkward in angles.
3. It drops points with no neighbour at nearly the same distance, because real objects are solid and noise is alone. Two each side, so a single dropout next to a real point doesn't make it look lonely.
4. The rover sweeps a strip as wide as its body. A cone is too wide far away (false stops) and too narrow close up (misses).
5. So one stray point can't stop the rover, while two points (a thin leg) still can.
6. Two thresholds (30 cm stop, 40 cm resume) stop the decision flickering. It's lopsided because a missed stop (a collision) is far worse than a false one (0.7 s lost).
7. The bumper is what hits things. The LiDAR sits 8 cm behind it, so measuring from the LiDAR would stop 8 cm too late.
8. `ScanSource` is an interface; `virtual` lets `X2Source` or `SimSource` plug in without the rest of the code changing; `unique_ptr` deletes the object automatically.
9. Anything below or above its scan plane (a shoe, a cable), and glass or mirrors. The camera covers low objects.

14. Glossary

Term	Plain meaning
LiDAR	Light Detection And Ranging: measuring distance with a laser.
Triangulation	Finding distance from where a reflected dot lands on a sensor beside the laser.
Scan	One full 360° rotation's worth of readings (~430 on the X2).
Polar / Cartesian	(angle, distance) versus (x ahead, y left). Same point, two descriptions.
Radian	The angle unit maths uses: 180° = π ≈ 3.1416 radians.
Corridor	The rectangle of floor the rover's body is about to drive over.
Dropout	A reading with no return, reported as 0 m.
Outlier / lonely point	A reading that doesn't agree with its neighbours; usually noise.
k-th nearest	The k-th smallest distance; robust to a few stray points.

Hysteresis	Different thresholds for switching on and off, to prevent flickering.
Latch	Something that remembers its state until a clear reason to change.
Interface / virtual	A promise of which functions exist; different classes fulfil it differently.
Baud rate	Serial speed in bits per second; both ends must match.
Single-channel	One-way serial: the device talks and never listens.

Matches `lidar_stop.cpp` as of 30 Sep 2026 (lonely-point filter, kthNearest = 2, stop 0.30 m / resume 0.40 m).